Informe INF-239
Diseño Documental y Análisis Comparativo SQL vs NoSQL (SaludUSM)

Alumnos:
Lukas Labrín
Pablo Rodríguez

Profesor: Ricardo Salas

1. Diseño del Modelo Documental

Para migrar el sistema de SaludUSM a una arquitectura orientada a documentos, se realizó un analisis al modelo normalizado de la tarea 2 y se rediseñó priorizando los patrones de acceso principales: el comprobante de reserva y la ficha de atención

1.1. Estructura de Colecciones

El sistema se redujo a 7 colecciones principales, manteniendo los catalogos separados para preservar la consistencia de los datos:

1. Previsión (Catálogo)
2. especialidad (Catálogo)
3. estado_cita (Catálogo)
4. centro_medico (Catálogo)
5. paciente
6. medico
7. cita (Colección transaccional principal)

1.2. Justificación de Relaciones y Anidamiento

- Relación Cita - Atención (embebida): Tienen una relación 1:1 o 1:0 si no existe el paciente. Se decidió embeber el objeto “atención” directamente dentro del documento de la “cita” porque no existe un caso de uso en el sistema donde se consulte una atención sin requerir los datos de su cita origen.
- Atención - Diagnóstico / Receta (arreglos embebidos): Se estructuraron como arreglos de objetos dentro de la atención. Estos datos son dependientes existenciales del evento clínico, un medicamento recetado carece de contexto fuera de la atención específica donde se prescribió.
- Relación Paciente - Cita (Referenciado): se separaron en colecciones distintas mediantes id_paciente. Si las citas se anidaran dentro del paciente, el documento crecería de forma indefinida con los años (anti-patrón de arreglos desatados), arriesgando superar el limite de 16 Megabytes por documento de MondoDB.
- Relación Médico - Especialidad / Centro (Arreglos referenciados): El médico contiene arreglos de identificadores. Al ser entidades maestras de catálogo, referenciarlas evita la redundancia masiva. Si un centro cambia su nombre, se actualiza una única vez en su colección en lugar de modificar los documentos de todos los médicos que ejercen allí.

2. Análisis Comparativo: SQL vs. NoSQL

2.1. Modelo de Datos y esquema

En MySQL, el esquema es rígido y tabular. La entidad “Atención Clínica” requería fragmentar la información en cuatro tablas distintas (cita, atención, diagnostico_atención, receta_linea) para cumplir con la estructura bidimensional. En MongoDB (NoSQL) el esquema es flexible y orientada a documentos, lo que permitió consolidar toda la ficha clínica de una atención en un único documento JSON jerárquico, mejorando la localidad de los datos en el disco.

2.2. Normalización y redundancia

En MySQL, el diseño se forzó hasta la Tercera Forma Normal (3FN), eliminando casi toda redundancia mediante claves foráneas. En MongoDB, se aplicó una desnormalización controlada. Al embeber los nombres de los medicamentos y las descripciones de los

diagnósticos directamente en la cita, se introduce redundancia: si el nombre de un medicamento cambia oficialmente, habría que actualizar miles de documentos históricos. Para entidades críticas que cambian (como el nombre de un Médico o un Centro), se priorizó la normalización mediante referencias.

2.3. Integridad y reglas de negocio

MySQL delega la integridad referencial al motor de base de datos (eliminación en cascada, Foreign Keys) y utiliza Triggers para reglas complejas (por ejemplo, evitar agendar citas en el pasado o validar que el médico posea la especialidad). En MongoDB, el motor no posee Triggers relacionales nativos de la misma robustez; por lo tanto, la responsabilidad de validar las reglas de negocio (verificar fechas, especialidades y disponibilidad) se traslada por completo a la capa de aplicación (Python).

2.4. Lenguaje de consulta

En MySQL, la consulta del "Historial Clínico" es declarativa mediante SQL: se describen las tablas a unir mediante múltiples JOIN y el motor decide el plan de ejecución. En MongoDB, se utilizó MQL a través del Aggregation Pipeline. Este enfoque es más procedimental; se define un flujo de datos secuencial donde el documento pasa por etapas de filtrado ($lookup), desanidamiento ($$unwind$$) y proyección ($project).

2.5. Flexibilidad del esquema: Alergias y Signos Vitales

Si SaludUSM decide agregar alergias y signos vitales (campos que varían por especialidad):

En MySQL: Sería altamente disruptivo. Requeriría comandos DDL (ALTER TABLE) o la creación de nuevas tablas con nuevas claves foráneas, forzando a detener el sistema o lidiar con valores nulos (NULL) masivos para las atenciones antiguas.

En MongoDB: Gracias a su naturaleza schema-less, basta con que la aplicación comience a enviar los nuevos arreglos y objetos en las nuevas inserciones. Los documentos antiguos coexistirán sin problemas sin necesidad de scripts de migración estructural.

3. Conclusión y Recomendaciones Arquitectónicas

3.1. Tabla Resumen

Característica	MySQL	MongoDB
Esquema.	Rigido. Fuertemente Tipado.	Flexible. Dinámico.
Relaciones.	Forzadas vía	Referencias lógicas

	Foreign keys y JOINS.	o documentos embeidos.
Escalabilidad.	Vertical.	Horizontal nativa.
Integridad.	Motor de base de datos (ACID).	Capa de aplicación.
Redundancia.	Mínima(3fn).	Desnormalización intencional para lectura rápida

3.2. Recomendaciones por Módulo para SaludUSM

1.Módulo de Agendamiento (Reserva de horas):

Recomendación: MySQL (SQL).

Justificación: El agendamiento requiere transaccionalidad estricta (ACID) para evitar sobrecupos (dos personas tomando la misma hora simultáneamente) y controles estrictos de integridad referencial. El motor relacional es la herramienta natural para garantizar la exactitud operativa de los calendarios médicos.

2.Módulo de Ficha Clínica (Atenciones):

Recomendación: MongoDB (NoSQL).

Justificación: Los historiales clínicos rara vez cambian una vez emitidos y requieren alta flexibilidad. Un cardiólogo y un oftalmólogo registran variables totalmente distintas. MongoDB permite almacenar esta heterogeneidad de datos en un solo documento sin alterar el esquema global del sistema.

3.Panel de Gestión (Estadísticas y Tasa de Inasistencia):

Recomendación: MongoDB (Replica Set Secundario).

Justificación: El Aggregation Pipeline demostró ser una herramienta muy potente para generar estadísticas de forma procedimental. Al aprovechar la arquitectura de Replica Set, los paneles de gestión pueden conectarse a leer de los nodos secundarios sin afectar el rendimiento transaccional del nodo primario.